

Implementation_Phases_md

AI Test Case Generator — Phase-wise Implementation Plan (Learner Edition)

This plan turns the finalized architecture (`architecture.md`) into a staged, buildable roadmap for a **single developer learning RAG**. It runs locally, uses MongoDB Atlas M0 for hybrid search, and adds capability one layer at a time so you always have a working app after each phase.

1. Delivery Principles

- Ship something runnable after every phase (never a big-bang build).
- Build the RAG core first; treat review/export/polish as later layers.
- Keep AI providers behind adapters from day one (embeddings + LLM).
- Enforce `projectId` scoping everywhere from the start.
- Prefer transparent, debuggable techniques over "smart but opaque" ones.
- Define clear exit criteria before moving to the next phase.

2. Phase Overview

| Phase | Focus | Primary Outcome |

|---|---|---|

| Phase 0 | Setup & skeleton | Monorepo, config, DB connection, health check |

| Phase 1 | Ingestion pipeline | Upload → chunk → embed → store, with async jobs |

| Phase 2 | Retrieval engine | Hybrid BM25 + vector search, scoped + preprocessed |

| Phase 3 | Generation | Validated, cited test cases + confidence score |

| Phase 4 | Review, export & polish | Review flow, JSON/CSV/Excel export, logging, docs |

Maps directly to architecture decisions Q1–Q20.

3. Phase 0 — Project Setup & Skeleton

Goal

Establish the local engineering baseline so feature work has no ambiguity.

Scope

- Create the TypeScript monorepo (`apps/web`, `apps/api`, `packages/\*`) per architecture §18.
- Set up tooling: TypeScript, ESLint/Prettier, npm scripts, basic test runner.
- Implement the **typed config module** (Q19): load `.env`, validate required keys with Zod, fail fast. Commit `.env.example`, git-ignore `.env`.
- Connect to **MongoDB Atlas M0** (Q4) and verify read/write.

- Add the **correlation ID + structured logging** middleware (Q18).
- Add `GET /health/live` and `GET /health/ready`.
- Stub the React app with a project selector shell.

Deliverables

- Bootable API and web dev servers.
- Verified Atlas connection.
- Typed config with startup validation.
- Health endpoints and structured logging.

Exit Criteria

- `npm run dev` starts API + web without errors.
- Missing env vars cause a clear startup failure.
- `/health/ready` reports the DB as reachable.

4. Phase 1 — Ingestion Pipeline (Q3, Q5, Q7, Q8)

Goal

Turn uploaded sources into stored, embedded chunks via an async job.

Workstreams

1. **Projects & data model**
 - `projects`, `source_documents`, `chunks`, `ingestion_jobs` collections (architecture §12).
 - Compound indexes starting with `projectId`; unique `(projectId, sourceId, version)`; chunk `contentHash` index.
2. **Uploads** (Q3)
 - `POST /uploads` with type/size validation for PDF, Excel, Markdown/text, Swagger/OpenAPI.
3. **Extractors** (`packages/extractors`)
 - PDF text, Excel sheets/rows, Markdown, and OpenAPI operation parsing.
4. **Chunking** (Q7, `packages/chunking`)
 - Structure-aware split (headings / rows / one chunk per OpenAPI operation), then semantic split for long sections, with overlap. Compute `contentHash` per chunk.
5. **Embedding adapter** (Q5, `packages/embeddings`)
 - Interface `embed(texts): number[][] + dimension`; default Mistral, local model fallback.
6. **Async ingestion jobs** (Q8)
 - `POST /ingestions` returns `202` + `jobId`; background task runs `queued` → `processing` → `done/failed`; `GET /ingestions/:jobId` for polling; retry endpoint.
7. **Vector index**
 - Create the Atlas Vector Search index matching the adapter's dimension.

Deliverables

- End-to-end ingestion for all four source types.

- Chunks + vectors stored and indexed.
- Pollable job status with counts/errors.

Exit Criteria

- A PDF/Markdown/Excel file and a Swagger spec each ingest successfully via a job.
- Chunks appear in MongoDB with embeddings and citations to source.
- Re-ingesting the same source version does not duplicate chunks.

5. Phase 2 — Retrieval Engine (Q4, Q9, Q10, Q11, Q16)

Goal

Given a query, return the most relevant, deduplicated, cited evidence — scoped to a project.

Workstreams

1. **Query preprocessing** (Q10)
 - Normalize; expand abbreviations/synonyms from the `abbreviations` dictionary. Add `GET/PUT /abbreviations`.
2. **Hybrid search** (Q9)
 - BM25 (Atlas full-text) + vector search; **score fusion** into one ranked list.
 - Enforce `projectId` filter server-side; support optional filters (source type, release, tags).
3. **Context assembly** (Q11)
 - Deduplicate near-identical chunks (hash/threshold), take top-K, apply a token/character budget, preserve citations (Q16).
4. **APIs**
 - `POST /retrieval/search` (preview/debug) and `GET /evidence/:chunkId`.

Deliverables

- Working hybrid retrieval with fusion, dedup, and citations.
- Query expansion dictionary editable via API.

Exit Criteria

- A query returns fused BM25 + vector results, ranked and deduplicated.
- Results never cross `projectId` boundaries.
- Each result carries a source citation and relevance score.

6. Phase 3 — Test-Case Generation (Q6, Q12, Q13, Q15)

Goal

Generate structured, validated, cited test cases with a confidence score.

Workstreams

1. ****LLM adapter**** (Q6, `packages/llm`)
 - Interface `generate(prompt)`; adapters for Groq (default), OpenAI, Anthropic; optional Ollama. Timeouts + one retry.
2. ****Generation controls**** (Q13)
 - Accept test types, coverage level, priority, source filters, output format on `POST /generations`.
3. ****Prompt assembly**** (`packages/prompts`)
 - Instructions + query + cited evidence + controls; versioned prompt template.
4. ****Schema validation**** (Q12)
 - Strict JSON Schema (Zod/Ajv) for the test-case shape; one bounded repair retry; mark `failed` if still invalid (store nothing broken).
5. ****Confidence score**** (Q15, `packages/confidence`)
 - Composite of retrieval relevance (40%), requirement coverage (30%), output completeness (30%) → High/Medium/Low.
6. ****Persistence & APIs****
 - Store in `test\_cases` (+ `generation\_requests`); expose `GET /generations/:id`, `GET /test-cases`, `GET /test-cases/:id`, `/confidence`, `/generation-options`, `regenerate`.

Deliverables

- Generation pipeline producing schema-valid, cited test cases.
- Confidence score + band on each test case.
- Swappable LLM provider via config.

Exit Criteria

- A generation request returns test cases that pass JSON Schema validation.
- Each test case includes evidence citations and a confidence band.
- Switching the LLM provider requires only a config change.
- Malformed model output triggers exactly one repair, then fails cleanly.

7. Phase 4 — Review, Export & Polish (Q14, Q17, Q18)

Goal

Add the human loop, get results out, and finish the app for demo/use.

Workstreams

1. ****Single-step review**** (Q14)
 - `draft` → `approved/rejected` with comment; edits create a `test\_case\_versions` entry; `GET /test-cases/:id/versions`.
2. ****Export**** (Q17)
 - `POST /exports` → JSON and CSV/Excel (citations preserved); `GET /exports/:id` for download.

3. ****Frontend completion****

- Full flow in the UI: project → upload → ingestion status → generate → review/edit/approve → export.

4. ****Observability & docs**** (Q18)

- Confirm correlation IDs flow through ingestion/generation logs.
- Write `README.md` (setup, env vars, running locally, Atlas index setup) and finalize the OpenAPI file.

Deliverables

- Working review lifecycle with version history.
- JSON + CSV/Excel export.
- Complete, demoable UI and documentation.

Exit Criteria

- A generated test case can be edited, approved, or rejected with a comment.
- Approved/selected test cases export to JSON and CSV/Excel.
- A single request is traceable across logs via its correlation ID.
- A new user can set up and run the app from the README alone.

8. Phase Dependencies

- Phase 0 → all others (baseline).
- Phase 1 must store embedded chunks before Phase 2 can search.
- Phase 2 must return cited evidence before Phase 3 can generate grounded output.
- Phase 4 depends on Phase 3 producing test cases to review and export.

9. Suggested Learning Milestones

- ****Milestone A (after Phase 1):**** "I can upload a Swagger spec and see chunked, embedded content in MongoDB."
- ****Milestone B (after Phase 2):**** "I can query and get fused, deduplicated, cited evidence scoped to a project."
- ****Milestone C (after Phase 3):**** "I can generate schema-valid test cases with citations and a confidence score, and swap the LLM by config."
- ****Milestone D (after Phase 4):**** "I can review, approve, and export test cases, and trace any request in the logs."

10. Optional Enhancements (Post-MVP)

Aligned with architecture §20 "future" items — build only after the core works:

- LLM-based query rewriting (on top of dictionary expansion).

- Summarization/compression in context assembly (target end-state for Q11).
- Reranking after hybrid fusion.
- Coverage-matrix traceability view (step up from citations).
- Move ingestion to a separate worker + queue (Pub/Sub / BullMQ) — API contract already supports it.
- Two-stage review + audit events.
- Metrics/tracing dashboards; token-cost tracking.
- Voice extension (input only, or full two-way).

11. Recommendation

Follow the phases in order and treat each exit criterion as a gate. This keeps the project always-runnable, front-loads the RAG concepts that carry the most learning value (ingestion → hybrid retrieval → validated generation), and leaves every enterprise feature as a clean, optional upgrade rather than a rewrite.